How We Extract and Process Each Medical PDF — Plain Language Guide

The Big Picture: What Are We Even Doing?

We have 10 medical PDFs — clinical guidelines written by doctors for other doctors. Our goal is to teach an AI chatbot what's inside them, so it can answer a patient's diabetes questions correctly.

But you can't just hand a PDF to an AI. PDFs are designed for human eyes — two-column layouts, fancy tables, footnotes, headers, page numbers everywhere. An AI needs plain, structured text with labels attached.

So we run each PDF through an extractor — a script that reads the PDF and produces a clean .md (markdown) file. That .md file is what the AI will eventually learn from.

Step 1 — Reading the PDF

We use two different tools depending on the PDF's complexity:

Docling (the smarter, slower tool)

Used when a PDF has complex two-column layouts, tables that span multiple rows, or text that a simple reader would scramble. Docling has a visual understanding of the page — it can see that column 1 text should not be mixed with column 2 text.

Sources using Docling: ADA 2026, Anoop Misra, ICMR STW 2024, ESC 2023, WHO HEARTS, Telemedicine Guidelines.

pdfplumber (the simpler, faster tool)

Used when a PDF is straightforward or when Docling crashes. For three PDFs (ICMR-NIN food tables, IDF-DAR Ramadan, KDIGO kidney disease), Docling ran out of computer memory and stopped mid-document. pdfplumber reads page by page without loading everything at once, so it handles these large/dense PDFs without crashing.

Sources using pdfplumber: RSSDI 2022, ICMR-NIN food tables, IDF-DAR Ramadan, KDIGO 2022.

Why does this happen at the PDF format level, not our design?

The three PDFs that caused memory crashes are either very long (333 pages, 585 pages, 128 pages) or have extremely dense tables on every page. Docling tries to visually render every page before extracting — on a laptop CPU with limited RAM, that rendering process runs out of memory. This is a hardware/PDF-complexity problem, not a flaw in our system. pdfplumber sidesteps it by reading only text, not rendering.

Step 2 — Converting Tables

PDFs store tables in a way that a simple text reader garbles — cells merge in the wrong order, column headers end up in the middle of data rows, or the same cell appears twice.

After extracting raw text, we rebuild every table manually. We go cell by cell, track which cells span multiple rows (so we don't repeat their content), and render a clean grid:

```
| HbA1c Target | Patient Type      | Evidence Grade |
|--------------|-------------------|----------------|
| < 7.0%       | Most adults       | A              |
| < 8.0%       | Elderly/frail     | B              |
```

This is purely a formatting fix. The content is unchanged — we're just making it readable.

Step 3 — The Document Header (the first block in every parsed file)

Open any parsed .md file and the very first thing you see is a block like this:

```
<!-- rag_metadata
  source: RSSDI_2022
  title: RSSDI Clinical Practice Recommendations...
  year: 2022
  population: Adult T2DM patients, India
  retrieval_tier: core
  india_specific: true
-->
```

The <!-- ... --> means it is a comment — invisible to humans reading normally, but the AI system reads it.

This is the document header — a label we attach to the whole document. It tells the AI:

- Where this came from (source: RSSDI_2022)
- When it was written (year: 2022)
- Who it is for (population: Adult T2DM patients, India)
- When to use it (retrieval_tier: core = use on every question; triggered = use only for heart/kidney/Ramadan questions)
- Is it India-specific? (india_specific: true/false) — RSSDI and ICMR are calibrated for Indian physiology; ADA and ESC are global guidelines used as backup

Why bother? Because without this label, the AI treats all 10 documents equally. With it, the AI knows: for a standard blood sugar question, check RSSDI first. For a heart disease question, check ESC. For a Ramadan fasting question, check IDF-DAR.

Step 4 — Section Metadata (labels on each chapter)

After the document header, inside the document, every major section heading gets a smaller label:

```
## Glycaemic Targets
<!-- rag_metadata source=RSSDI_2022 section="Glycaemic Targets"
     topic_tags="glycemic_targets, HbA1c" population="T2DM India" year=2022 -->
```

This labels which part of the document a piece of text comes from. Later, when we split the document into small chunks for the AI, each chunk inherits this label. So even a 3-sentence chunk knows it is about "glycaemic targets from RSSDI 2022 for Indian adults."

Why is this important? When a patient asks "what should my HbA1c be?", the AI searches for chunks tagged glycemic_targets. Without these labels, it would have to read every sentence in all 10 documents to find the answer.

Step 5 — Evidence Grade Annotations (RSSDI-specific)

What is an evidence grade?

A clinical guideline is written by a committee of doctors. Every recommendation they make is rated by how strong the proof is:

Grade	Meaning in plain words
A	Proven by multiple large, well-run clinical trials. High confidence.
B	Proven by one good study or consistent smaller studies. Moderate confidence.
C	Based on smaller studies or mixed results. Lower confidence.
E	Expert opinion only — no strong trial data exists yet.

In the RSSDI document, these grades appear in brackets right after a recommendation, like:

> *"Metformin remains the preferred first-line agent in most patients with T2DM. (A)"*

Our extractor detects these grade markers and adds a label before that line:

```
<!-- rag_metadata source=RSSDI_2022 evidence_grade=A -->
Metformin remains the preferred first-line agent in most patients with T2DM. (A)
```

Why? So that for safety-critical questions ("what is the recommended first medication?"), the AI can be told to prefer Grade A chunks over Grade C chunks. A Grade A recommendation is not an opinion — it is backed by the strongest clinical evidence available.

Step 6 — ESC Evidence Class Annotations (ESC 2023-specific)

The European heart guidelines (ESC) use a different grading system. Every recommendation has two labels:

Class (how strongly recommended):

- Class I = Do this. It is recommended.
- Class IIa = This should be considered.
- Class IIb = This may be considered (weaker).
- Class III = Do NOT do this — it is harmful or ineffective.

Level (quality of evidence):

- Level A = Multiple randomised clinical trials (the gold standard of medical research)
- Level B = One good trial or a large patient registry
- Level C = Expert consensus, no strong trial data

So a recommendation might say: "Class I, Level A" — meaning: this is strongly recommended AND backed by the best evidence.

Our extractor scans every table in the ESC document. When a table has "Class" and "Level" columns, it labels that entire table block so the AI can filter: "show me only Class I, Level A recommendations about heart failure in diabetics."

Why is this done at our system level, not built into the PDF? The PDF has this information visually in tables — a doctor reading it can see it. But the AI cannot "see" tables the same way. We have to teach it explicitly what the structure means and tag it.

Step 7 — Safety Redline Annotations (IDF-DAR Ramadan-specific)

Some information is so safety-critical that the AI must never lose track of it, no matter how it is split up later. For the Ramadan fasting guidelines, specific blood sugar thresholds determine whether a patient should break their fast:

> *"Break fast immediately if blood glucose drops below 70 mg/dL or rises above 300 mg/dL."*

Our extractor finds these exact numbers and marks them:

```
<!-- rag_metadata safety_redline=true chunk_note=zero_loss_standalone_node -->
Break fast if BG < 70 mg/dL or > 300 mg/dL
```

safety_redline=true means: this chunk must always be retrieved intact when Ramadan is in context. It cannot be dropped or diluted by other results.

chunk_note=zero_loss_standalone_node is an instruction to the chunking step: do not split this sentence across two chunks. Keep it whole.

Step 8 — Meal-Timing Annotations (IDF-DAR-specific)

This is subtle but critical. In Ramadan fasting, there are two meal times:

- Suhoor/Sehri = the pre-dawn meal (before the fast begins)
- Iftar = the meal that breaks the fast at sunset

The dose adjustment advice for Suhoor is different from the advice for Iftar. If the AI mixes them up, a patient could take the wrong dose at the wrong time.

Our extractor finds every sentence that mentions a specific meal time alongside a drug class name, and labels it:

```
<!-- rag_metadata meal_context=suhoor -->
Reduce sulphonylurea dose at Suhoor to avoid pre-dawn hypoglycemia.
```

This way, a question about Iftar will never accidentally surface Suhoor advice.

Step 9 — Treatment Ladder Annotations (WHO HEARTS, ICMR STW 2024)

Some guidelines describe a step-by-step treatment escalation:

- Step 1: Start with Drug A
- Step 2: If not controlled, add Drug B
- Step 3: If still not controlled, add Drug C

If the AI sees only Step 2 without Step 1, it might give incomplete or dangerous advice. Our extractor finds every "Step N" line and labels it:

```
<!-- rag_metadata chunk_note=keep_atomic_large_window -->
Step 2: Add Telmisartan 40mg if target not reached on Amlodipine alone.
```

keep_atomic_large_window tells the chunker: keep this entire step sequence together — do not break it apart at token limits.

What Does a Parsed File Actually Contain? Reading it Top to Bottom

Here is what you will see in every parsed .md file, in order:

Section	What it is
<!-- rag_metadata ... --> block at the very top	Document-level label — who wrote it, when, for whom, when to use it
Plain English header	Human-readable summary: source, citation, population, scope
Clinical priority note (blockquote >)	Instructions to the AI: prefer this source / only use this when triggered
--- horizontal rule	Divider between our added header and the original PDF content
Original document content	The actual PDF text, converted to markdown
<!-- rag_metadata ... --> comments scattered throughout	Section labels, evidence grade labels, safety labels — all added by us
Tables	Rebuilt from PDF table structures, clean grid format
<!-- page N --> markers (in pdfplumber extracts)	Page number markers so the AI knows where in the original PDF a chunk came from

Everything inside <!-- ... --> was added by us — it is not in the original PDF. Everything outside those tags is the original clinical content.

Why Some Things Come From PDF Format, Others From Our Design

Issue	Root cause
pdfplumber used instead of Docling for 3 PDFs	PDF format — those PDFs are too large/dense for Docling's visual renderer on a standard CPU
Tables look scrambled before we fix them	PDF format — PDFs store tables as positioned boxes, not rows and columns
Author lists / copyright pages in the output	PDF format — those pages exist in the PDF; we extract everything including front matter
<!-- rag_metadata --> comments throughout	Our system design — these are our labels, not from the PDF
Evidence grade labels (A), (B)	Both — the grades come from the PDF; we detect and re-label them for the AI

Safety redline markers

Our system design — the thresholds come from the PDF; the safety_redline=true tag is ours

keep_atomic_large_window instructions

Our system design — a decision about how to chunk content later

The Flow From PDF to AI Response

```
PDF
  ↓  (Docling or pdfplumber reads it)
Raw text + tables
  ↓  (we rebuild tables, fix encoding)
Clean markdown
  ↓  (we add rag_metadata labels throughout)
Annotated .md file  ← this is what you see in the parsed/ folder
  ↓  (next step — not yet built)
Split into chunks (~300–500 tokens each)
  ↓
Each chunk gets a structured metadata tag (source, year, topic, tier, grade)
  ↓
Uploaded into Qdrant vector database
  ↓
Patient asks a question
  ↓
AI searches database, finds top 20 matching chunks
  ↓
Reranker picks the best 5
  ↓
AI generates an answer grounded in those 5 chunks
```

The .md files in parsed/ are the output of step 3. Everything before them (PDF reading, table fixing, labelling) is complete. Everything after them (chunking, embedding, vector database) is the next engineering milestone.

Quick Reference: Which Document, What Labels, Why

Document	Key labels added	Why
RSSDI 2022	evidence_grade=A/B/C/E per recommendation	So AI prefers strongest-evidence advice
ICMR STW 2024	treatment_algorithm, Step N labels	Keep escalation steps together
ADA 2026	Section separators between 15 sub-PDFs	Merged into one file; separators track which sub-document
Anoop Misra	≥ symbol restoration, Indian food glossary	PDF encoded ≥ as a non-standard character; 35 Hindi/regional food terms mapped to English
ICMR-NIN	Food group headers (Cereals, Legumes, Fish...)	7,000+ food rows grouped so queries like "carbs in matta rice" find the right group
ESC 2023	Class I/IIa/IIb/III + Level A/B/C per recommendation table	Enables evidence-class filtering for cardiology queries
IDF-DAR	safety_redline=true on BG thresholds; meal_context=suhoor/iftar	Prevent mixing up Suhoor/Iftar advice; never drop fast-breaking thresholds
KDIGO 2022	Page markers only; annotations deferred	Extraction complete; annotation decisions pending chunking strategy
WHO HEARTS	BP threshold safety labels, Step N ladder	

labels, HEARTS module tags

Keep BP decision nodes and step-up ladders

intact

Telemedicine

Drug list type (List O/A/B), consultation mode labels

Scope enforcement — AI uses this silently to know what it can/cannot do